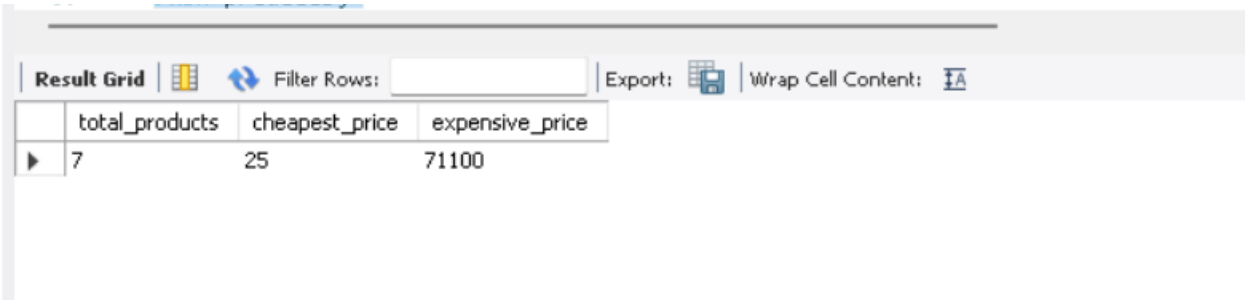
PRACTICE SET – 4

1. Total number of products, cheapest price, and most expensive price in one query.

```
83
84 •    SELECT COUNT(*) AS total_products ,
85          MIN(price) AS cheapest_price ,
86          MAX(price) AS expensive_price
87      FROM products;
88
```

DATABASE OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_products	cheapest_price	expensive_price
7	25	71100

2. Total stock of all products, and total stock of Electronics only.

```
90 •    SELECT SUM(stock) as total_stock ,
91          SUM(
92              CASE
93                  WHEN category = "Electronics" THEN stock
94                  ELSE 0
95              END
96              ) AS electronic_stock
97      FROM products ;
```

DATABASE OUTPUT:



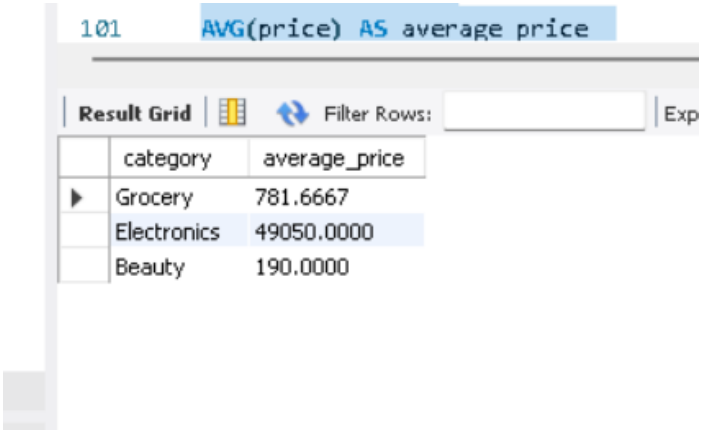
Result Grid | Filter Rows: | Export:

total_stock	electronic_stock
755	35

3. Average price per category (GROUP BY)

```
 99
100 •   SELECT category ,
101     AVG(price) AS average_price
102     FROM products
103     GROUP BY category ;
104
```

DATABASE OUTPUT:

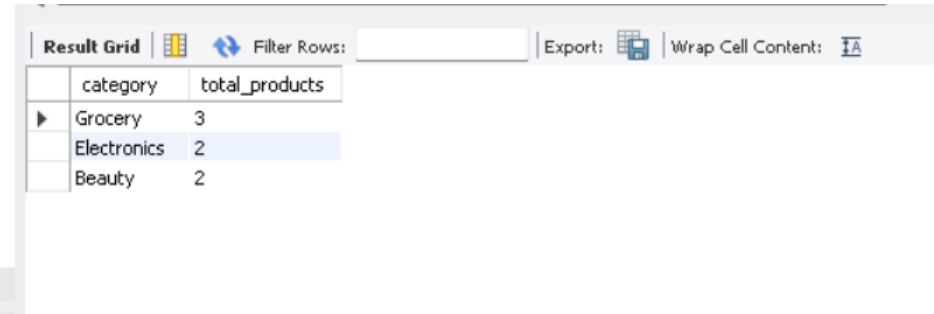


category	average_price
Grocery	781.6667
Electronics	49050.0000
Beauty	190.0000

4. Count products per category, but show only categories with at least 2 products. WHERE or HAVING? Why?

```
106
107 •   SELECT category ,
108     COUNT(*) AS total_products
109     FROM products
110     GROUP BY category
111     HAVING COUNT(*) >=  2 ;
112
```

DATABASE OUTPUT:



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

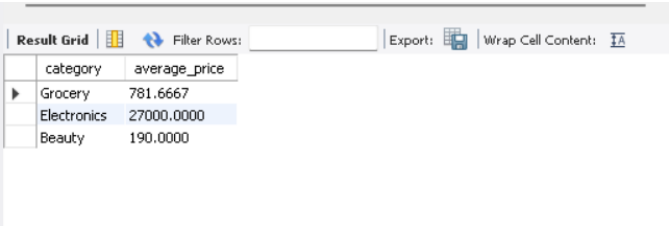
category	total_products
Grocery	3
Electronics	2
Beauty	2

5. Average price per category, counting only products cheaper than 60000.

Which filter goes in WHERE?

```
113
114 •   SELECT category ,
115     AVG(price) AS average_price
116     FROM products
117     WHERE price < 60000
118     GROUP BY category ;
119
```

DATABASE OUTPUT:



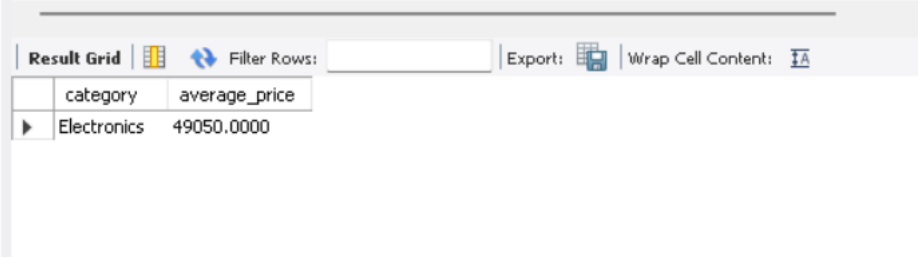
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	average_price
Grocery	781.6667
Electronics	27000.0000
Beauty	190.0000

6. Categories where the average price is above 1000, sorted biggest first. Use the full general order.

```
119
120 •   SELECT category ,
121     AVG(price) AS average_price
122     FROM products
123     GROUP BY category
124     HAVING AVG(price) > 1000
125     ORDER BY AVG(price) DESC ;
126
```

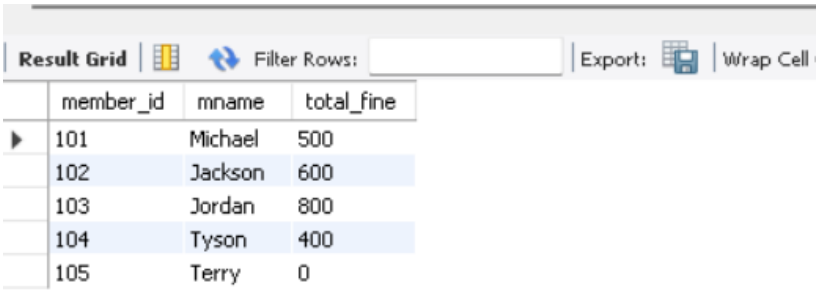
DATABASE OUTPUT:



7. Library: total fine per member (join + GROUP BY). Members with no loans must show 0. Which join?

```
109 •   SELECT members.member_id, members.mname ,
110     COALESCE(SUM(loans.fine),0) AS total_fine
111     FROM members
112     LEFT JOIN loans
113     ON members.member_id = loans.member_id
114     GROUP BY members.member_id, members.mname;
115
```

DATABASE OUTPUT:



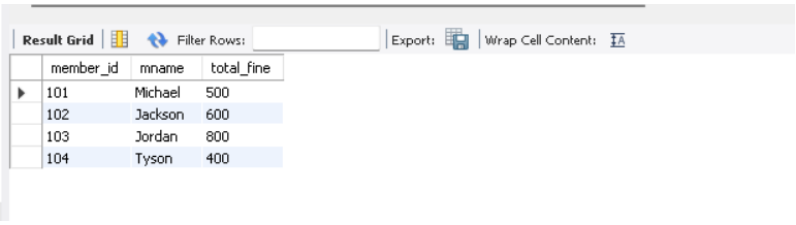
Result Grid | Filter Rows: | Export: | Wrap Cell

member_id	mname	total_fine
101	Michael	500
102	Jackson	600
103	Jordan	800
104	Tyson	400
105	Terry	0

8. Library: members whose total fine is above 100 (HAVING)

```
116
117 •   SELECT members.member_id , members.mname, SUM(loans.fine) AS total_fine
118     FROM members
119     INNER JOIN loans
120     ON members.member_id = loans.member_id
121     GROUP BY members.member_id , members.mname
122     HAVING SUM(loans.fine) > 100 ;
123
```

DATABASE OUTPUT:



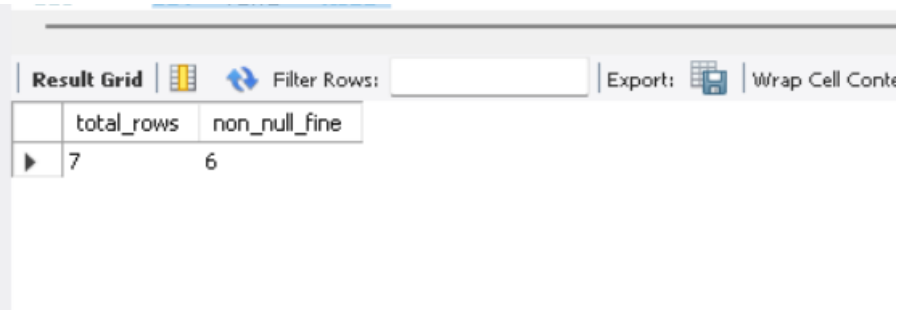
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

member_id	mname	total_fine
101	Michael	500
102	Jackson	600
103	Jordan	800
104	Tyson	400

9. COUNT(*) vs COUNT(fine): make one fine NULL and show the difference.

```
123
124 •   UPDATE loans
125     SET  fine = NULL
126      WHERE loan_id = 1 ;
127
128 •    SELECT COUNT(*) AS total_rows ,
129     COUNT(fine) AS non_null_fine
130     FROM loans ;
131
```

DATABASE OUTPUT:



total_rows	non_null_fine
7	6

10. In one sentence: why can HAVING see COUNT(*) but WHERE cannot?

WHERE filters individual rows before rows are grouped and aggregated, whereas HAVING filters the results after aggregate functions like COUNT(*) have computed the group summaries